

Simplified structured bindings protocol with pack aliases

Document #: P2120R0
Date: 2020-02-17
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction and Motivation](#)

[2 Proposal](#)

[3 Wording](#)

[4 References](#)

§ 1 Introduction and Motivation

When [[P1858R1](#)] was presented to EWGI in Prague [[EWGI.Prague](#)], that group requested that the structured bindings extension in that proposal was split off into its own paper. This is that paper, and the original paper continues on as an R2 [[P1858R2](#)].

Assuming the original paper gets adopted, and we end up with facilities allowing both declaring packs and indexing into them, it becomes a lot easier to implement something like `tuple` and opt it into structured bindings support:

```
template <typename... Ts>
class tuple {
    Ts... elems;
public:
    template <size_t I>
    constexpr auto get() const& -> Ts...[I] const& {
        return elems...[I];
    }
};

template <typename... Ts>
struct tuple_size<tuple<Ts...>>
    : integral_constant<size_t, sizeof...(Ts)>
{ };

template <size_t I, typename... Ts>
struct tuple_element<I, tuple<Ts...>> {
    using type = Ts...[I];
};
```

That’s short, easy to read, easy to write, and easy to follow - dramatically more so than the status quo without P1858.

But there’s quite a bit of redundancy there. And a problem with the tuple-like protocol here is that we need to instantiate a lot of templates. A declaration like:

```
auto [v1, v2, ..., vN] = tuple;
```

requires 2N+1 template instantiations: one for std::tuple_size, N for std::tuple_element, and another N for all the gets). That’s pretty wasteful. Additionally, the tuple-like protocol is tedious for users to implement. There was a proposal to reduce the customization mechanism by dropping std::tuple_element [P1096R0], which was... close. 13-7 in San Diego.

What do tuple_size and tuple_element do? They give you a number of types and then each of those types in turn. But we already have a mechanism in the language that provides this information more directly: we can provide a pack of types.

§ 2 Proposal

Currently, there are three kinds of types that can be used with structured bindings [P0144R2]:

- 1. Arrays (specifically T[N] and not std::array<T, N>).
- 2. Tuple-like: those types that specialize std::tuple_size, std::tuple_element, and either provide a member or non-member get().
- 3. Types where all of their members are public members of the same class (approximately).

This paper suggests extending the Tuple-like category by allowing types to opt-in by *either* providing a member pack alias named tuple_elements *or*, if not that, then the status quo of specialization both std::tuple_size and std::tuple_element.

In other words, a complete opt-in to structured bindings for our tuple would become:

With P1858	Proposed
<pre>template <typename... Ts> class tuple { Ts... elems; public: template <size_t I> constexpr auto get() const& -> Ts... [I] const& { return elems...[I]; } }; namespace std { template <typename... Ts> struct tuple_size<tuple<Ts...>></pre>	<pre>template <typename... Ts> class tuple { Ts... elems; public: using ...tuple_elements = Ts; template <size_t I> constexpr auto get() const& -> Ts...[I] const& { return elems...[I]; } };</pre>

With P1858	Proposed
<pre> : integral_constant<size_t, sizeof... (Ts)> { }; template <size_t I, typename... Ts> struct tuple_element<I, tuple<Ts...>> { using type = Ts...[I]; }; } </pre>	

This would also help those cases where we need to opt-in to the tuple protocol in cases where we do not even have a pack:

With P1858	Proposed
<pre> template <size_t> struct pair_get; template <typename T, typename U> struct pair { T first; U second; template <size_t I> constexpr auto get() const& -> decltype(auto) { return pair_get<I>::get(*this); } }; template <> struct pair_get<0> { template <typename T, typename U> static constexpr auto get(pair<T, U> const& p) -> T const& { return p.first; } }; template <> struct pair_get<1> { template <typename T, typename U> static constexpr auto get(pair<T, U> const& p) -> U const& { return p.second; } }; namespace std { </pre>	<pre> template <typename T, typename U> struct pair { T first; U second; using ...tuple_elements = tuple<T, U>::[:]; template <size_t I> constexpr auto get() const& -> tuple_elements...[I] const& { if constexpr (I == 0) return first; else if constexpr (I == 1) return second; } }; </pre>

With P1858	Proposed
<pre> template <typename T, typename U> struct tuple_size<pair<T, U>> : integral_constant<size_t, 2> { }; template <typename T, typename U> struct tuple_element<0, pair<T, U>> { using type = T; }; template <typename T, typename U> struct tuple_element<0, pair<T, U>> { using type = U; }; } </pre>	

Note that the whole `pair_get` implementation on the left can be replaced by introducing a pack alias as on the right anyway. And if that's already a useful thing to do to help implement a feature, it'd be nice to go that extra one step and make that already useful solution even more useful.

§ 3 Wording

Change 9.6 [dcl.struct.bind]/4:

Otherwise, if **either**

- (4.1) — the *qualified-id* `E::tuple_elements` names an alias pack, or
- (4.2) — the *qualified-id* `std::tuple_size<E>` names a complete class type with a member named `value`,

then the number and types of the elements are determined as follows. If in the first case, the number of elements in the *identifier-list* shall be equal to the value of `sizeof...`

`(E::tuple_elements)` and let T_i designate the type `E::tuple_elements...[i]`. Otherwise,

the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression **and**, the number of elements in the *identifier-list* shall be equal to the value of that expression, and let T_i designate the type `std::tuple_element<i, E>::type`. Let `i` be an index prvalue of type `std::size_t` corresponding to v_i . The *unqualified-id* `get` is looked up in the scope of `E` by class member access lookup ([basic.lookup.classref]), and if that finds at least one declaration that is a function template whose first template parameter is a non-type parameter, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)`, where `get` is looked up in the associated namespaces ([basic.lookup.argdep]). In either case, `get<i>` is interpreted as a *template-id*. [Note: Ordinary unqualified lookup ([basic.lookup.unqual]) is not performed. — end note] In either case, `e` is an lvalue if the type of the entity `e` is an lvalue reference and an xvalue otherwise. Given ~~the type~~ T_i

~~designated by `std::tuple_element<i, E>::type` and~~ the type U_i designated by either τ_i or $\tau_i\&\&$, where U_i is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise, variables are introduced with unique names r_i as follows:

```
S  $U_i$   $r_i$  = initializer ;
```

Each v_i is the name of an lvalue of type τ_i that refers to the object bound to r_i ; the referenced type is τ_i .

§ 4 References

[EWGI.Prague] EWGI. 2020. EWGI Discussion of P1858R1.

<http://wiki.edg.com/bin/view/Wg21prague/P1858R1SG17>

[P0144R2] Herb Sutter. 2016. Structured Bindings.

<https://wg21.link/p0144r2>

[P1096R0] Timur Doumler. 2018. Simplify the customization point for structured bindings.

<https://wg21.link/p1096r0>

[P1858R1] Barry Revzin. 2020. Generalized pack declaration and usage.

<https://wg21.link/p1858r1>

[P1858R2] Barry Revzin. 2020. Generalized pack declaration and usage.

<https://wg21.link/p1858r2>